Chapter 13 Question 10

Berk Bubuş - 21945939
Ahmet Uman - 2210356129

Our Question

10. Consider a very simple online auction system that works as follows. There are n *bidding agents*; agent i has a bid b_i, which is a positive natural number. We will assume that all bids b_i are distinct from one another. The bidding agents appear in an order chosen uniformly at random, each proposes its bid b_i in turn, and at all times the system maintains a variable b^* equal to the highest bid seen so far. (Initially b^* is set to 0.)

What is the expected number of times that b^* is updated when this process is executed, as a function of the parameters in the problem?

Example. Suppose $b_1 = 20$, $b_2 = 25$, and $b_3 = 10$, and the bidders arrive in the order 1, 3, 2. Then b^* is updated for 1 and 2, but not for 3.

Basically...

```
def find_max_and_count_updates(array):
    """Finds the maximum element in the array and counts how many times the max value
is updated."""
    if not array:
        return None, 0

    max_value = array[0]
    update_count = 0

    for element in array:
        if element > max_value:
            max_value = element
            update_count += 1
```

Analysis

- Since all bids are distinct, each bid b_i has potential to greatest number in previous numbers, update b^* . This means b_i is the new maximum.

- The question is when or how often a new maximum occur?

Analysis

- The probability that a particular bid b_i is the highest among the first *k* bids (for any *k*) is 1 / k.

- Given *n* agents appearing in random order, the probability that the i^{th} bid is a new maximum (and thus causes an update) is: 1 / i.

Expected Number of Updates

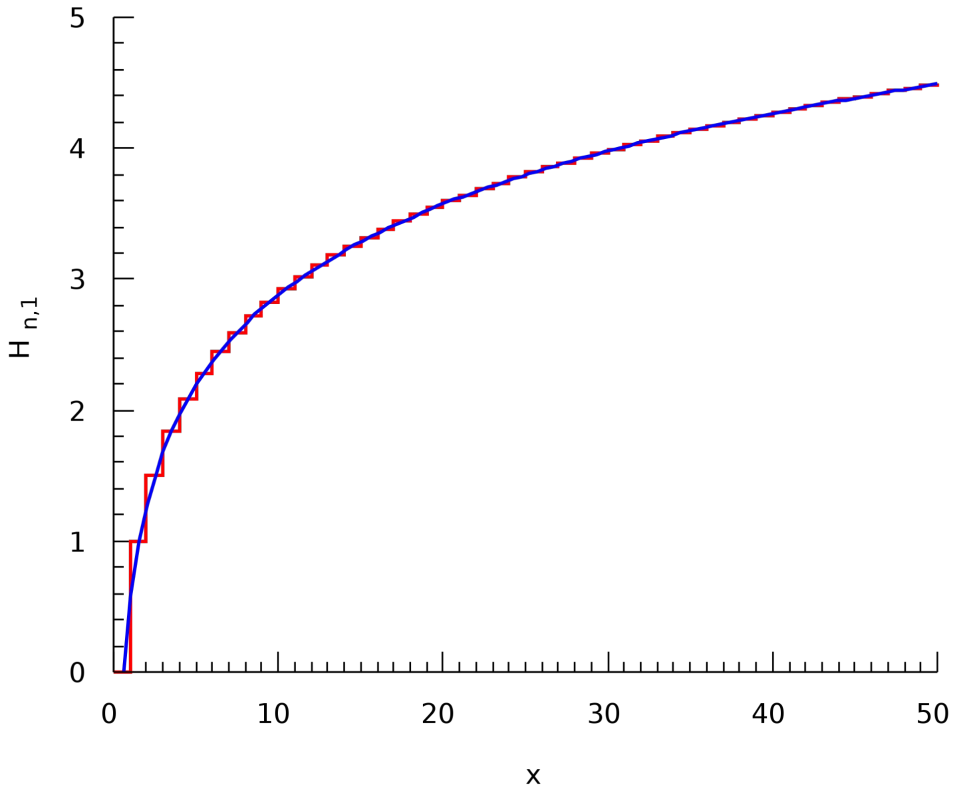
Thus, expected number of updates *E* is sum of the probabilities that each bid causes an update

Harmonic Number

Which also known as **Harmonic Number**. The n^{th} harmonic number H_n with n = ⌊x⌋ (red line) with its asymptotic limit **γ** + ln (x) (blue line) where **γ** is the **Euler–Mascheroni constant.**

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^{n} \frac{1}{k}.$$

Harmonic Number



The Euler-Mascheroni Constant

$$H_n = \ln(n) + \gamma$$

$$\gamma = \lim_{n\to\infty}\left(-\ln n + \sum_{k=1}^{n}\frac{1}{k}\right) \approx 0.577216.$$

The Code

```
def exact_result(n):
    res = 0
    for i in range(n):
        x = 1 / (i + 1)
        res += x
    return res
```

```
def hn(n):
    euler = 0.5772156649
    return np.log(n) + euler
```

The Example Table

	n	Exact Result	Hn	Real Life Result*
0	1	1.000000	0.577216	1
1	2	1.500000	1.270363	1.26
2	3	1.833333	1.675828	1.541
3	4	2.083333	1.963510	1.71
4	5	2.283333	2.186654	1.909
5	10	2.928968	2.879801	2.523
6	100	5.187378	5.182386	4.596
7	1000	7.485471	7.484971	7.046
8	10000	9.787606	9.787556	9.399
9	100000	12.090146	12.090141	11.827
10	1000000	14.392727	14.392726	14.35
11	10000000	16.695311	16.695311	16.9

Real Life Test Code

```
def average_update_counts(n, lower, num_trials=1000):
    """Runs the process num_trials times and calculates the average of update counts."""
    update_counts = []

    for _ in range(num_trials):
        random_array = create_random_array(n, lower, n)
        _, update_count = find_max_and_count_updates(random_array)
        update_counts.append(update_count)

    average_updates = sum(update_counts) / num_trials
    return average_updates
```